JOBSHEET 15

REGISTER USER DENGAN HASH PASSWORD & VALIDASI

PEMROGRAMAN BERBASIS FRAMEWORK

REIKA AMALIA SYAHPUTRI

2341720173

TI-3E / 19

TEKNOLOGI INFORMASI

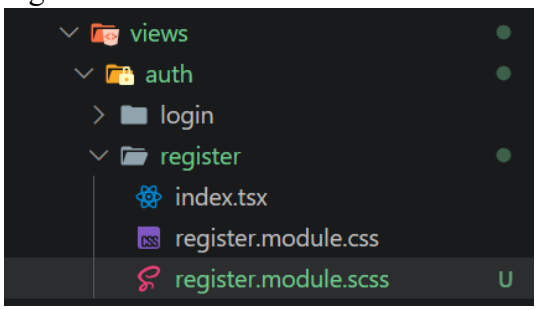
Github:

https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_15

C. Langkah Praktikum

Bagian 1 – Membuat Register View

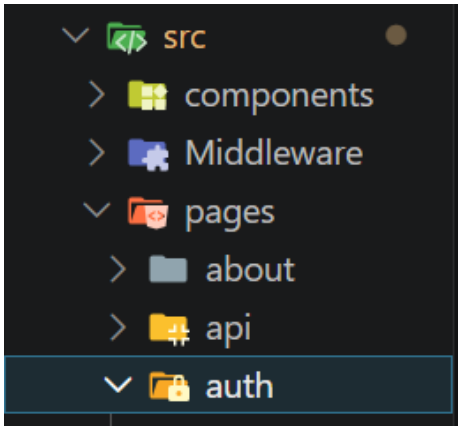
- Buat folder pada views dengan nama register dan tambahkan 2 file yaitu index.tsx dan register.module.scss



- Modifikasi file index.tsx (pada folder views/auth/register/index.tsx)

```
import Link from "next/link";
import style from "../../auth/register/register.module.scss";

const TampilanRegister = () => {

  return (
    <div className={style.register}>
      <h1 className={style.register__title}>Halaman Register</h1>
      <Link href="/auth/login">Ke Halaman Login</Link>
    </div>
  );
};

export default TampilanRegister;
```

- Buka file register.tsx pada folder auth/register.tsx



- Modifikasi file register.tsx (pada folder pages/auth/register.tsx)

```
1  import Link from "next/link";
2  import TampilanRegister from "@/views/auth/register";
3
4  const HalamanRegister = () => {
5    return (
6      <>
7        <TampilanRegister />
8      </>
9    );
10 };
11
12 export default HalamanRegister;
```

- Modifikasi register.module.scss

```css
.register{
    padding: 40px;
}

.register__title{
    font-size: 28px;
    font-weight: bold;
}
```

- Tambahkan form inputan pada file index.tsx (pada folder views/auth/register/index.tsx) Form berisi:
  Email

```
my-app-15 > src > views > auth > register > index.tsx > TampilanRegister
4   TampilanRegister = () => {
9
10  <div className={style.register__form}>
11    <form action="">
12
13      {/* Email */}
14      <div className={style.register__form__item}>
15        <label
16          htmlFor="email"
17          className={style.register__form__item__label}
18        >
19          Email
20        </label>
21        <input
22          type="email"
23          id="email"
24          name="email"
25          placeholder="Email"
26          className={style.register__form__item__input}
27        />
28      </div>
```

Full Name

```
my-app-15 > src > views > auth > register > index.tsx > TampilanRegister
4   TampilanRegister = () => {
29
30      {/* Fullname */}
31      <div className={style.register__form__item}>
32        <label
33          htmlFor="Fullname"
34          className={style.register__form__item__label}
35        >
36          Fullname
37        </label>
38        <input
39          type="text"
40          id="Fullname"
41          name="Fullname"
42          placeholder="Fullname"
43          className={style.register__form__item__input}
44        />
45      </div>
```

Password

```
my-app-15 > src > views > auth > register > index.tsx > TampilanRegister
4   TampilanRegister = () => {
46
47      {/* Password */}
48      <div className={style.register__form__item}>
49        <label
50          htmlFor="Password"
51          className={style.register__form__item__label}
52        >
53          Password
54        </label>
55        <input
56          type="password"
57          id="Password"
58          name="Password"
59          placeholder="Password"
60          className={style.register__form__item__input}
61        />
62      </div>
```

Button Register

```
my-app-15 > src > views > auth > register > index.tsx > TampilanRegister
4   TampilanRegister = () => {
63
64      {/* Button Register */}
65      <button
66        type="submit"
67        className={style.register__form__item__button}
68      >
69        Register
70      </button>
71
72    </form>
```

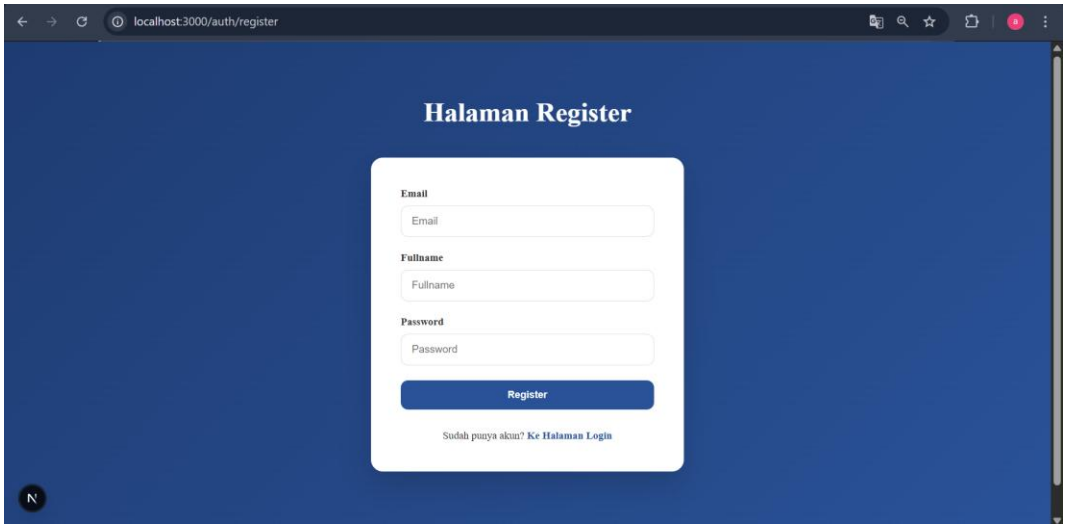
- Kode keseluruahannya sebagai berikut :

```python
import Link from "next/link";
import style from "../../auth/register/register.module.scss";

const TampilanRegister = () => {

  return (
    <div className={style.register}>
      <h1 className={style.register__title}>Halaman Register</h1>

      <div className={style.register__form}>
        <form action="">

          {/* Email */}
          <div className={style.register__form__item}>
            <label
              htmlFor="email"
              className={style.register__form__item__label}
            >
              Email
            </label>
            <input
              type="email"
              id="email"
              name="email"
              placeholder="Email"
              className={style.register__form__item__input}
            />
          </div>

          {/* Fullname */}
          <div className={style.register__form__item}>
            <label
              htmlFor="Fullname"
              className={style.register__form__item__label}
            >
              Fullname
            </label>
            <input
              type="text"
              id="Fullname"
              name="Fullname"
              placeholder="Fullname"
              className={style.register__form__item__input}
            />
          </div>

          {/* Password */}
          <div className={style.register__form__item}>
            <label
              htmlFor="Password"
              className={style.register__form__item__label}
            >
              Password
            </label>
            <input
              type="password"
              id="Password"
              name="Password"
              placeholder="Password"
              className={style.register__form__item__input}
            />
          </div>

          {/* Button Register */}
          <button
            type="submit"
            className={style.register__form__item__button}
          >
            Register
          </button>

        </form>

        {/* Halaman login */}
        <br />
        <p className={style.register__form__item__text}>
          Sudah punya akun? <Link href="/auth/login">Ke Halaman Login</Link>
        </p>
      </div>
      {/* Form selesai disini */}

    </div>
  );
};

export default TampilanRegister;
```

- Modifikasi register.module.scss

```scss
.register {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  background: linear-gradient(135deg, #1e3c72, #2a5298);
  padding: 20px;

  &__title {
    width: 100%;
    text-align: center;
    font-size: 36px;
    font-weight: 700;
    color: #ffffff;
    margin-bottom: 40px;
  }

  &__form {
    background: #ffffff;
    width: 100%;
    max-width: 420px;
    padding: 40px;
    border-radius: 16px;
    box-shadow: 0 20px 40px rgba(0, 0, 0, 0.15);

    &__item {
      display: flex;
      flex-direction: column;
      margin-bottom: 20px;

      &__label {
        font-size: 14px;
        font-weight: 600;
        margin-bottom: 8px;
        color: #333;
      }

      &__input {
        padding: 12px 14px;
        border-radius: 10px;
        border: 1px solid #ddd;
        font-size: 14px;
        transition: all 0.3s ease;

        &:focus {
          outline: none;
          border-color: #2a5298;
          box-shadow: 0 0 3px rgba(42, 82, 152, 0.2);
        }
      }

      &__button {
        width: 100%;
        padding: 12px;
        border-radius: 10px;
        border: none;
        background: #2a5298;
        color: #ffffff;
        font-weight: 600;
        cursor: pointer;
        transition: 0.3s ease;

        &:hover {
          background: #1e3c72;
          transform: translateY(-2px);
        }
      }

      &__text {
        font-size: 14px;
        color: #333;
        text-align: center;
        margin-top: 8px;

        a {
          color: #2a5298;
          font-weight: 600;
          text-decoration: none;

          &:hover {
            text-decoration: underline;
          }
        }
      }
    }

    &__error {
      color: red;
      font-size: 14px;
      margin-top: 8px;
    }
  }
}

/* Animation */
@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(15px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* Responsive */
@media (max-width: 768px) {
  .register {
    &__form {
      padding: 30px 20px;
    }

    &__title {
      font-size: 26px;
    }
  }
}
```

- Jalankan browsernya http://localhost:3000/auth/register sehingga tampilan sebagai berikut

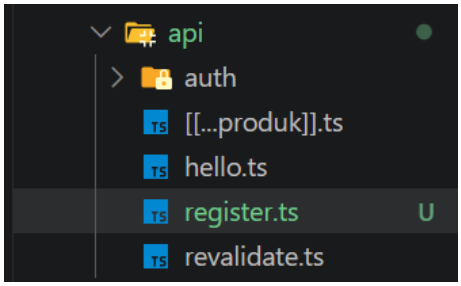


Bagian 2 – Membuat API Register

- Buka file servicefirebase.ts pada folder src/utils/db dan modifikasi

```typescript
import {
  getFirestore,
  collection,
  getDocs,
  getDoc,
  doc,
  query,
  addDoc,
  where,
} from "firebase/firestore";
import app from "./firebase";

const db = getFirestore(app);

export async function retrieveProducts(collectionName: string) {
  const snapshot = await getDocs(collection(db, collectionName));
  const data = snapshot.docs.map((doc) => ({
    id: doc.id,
    ...doc.data(),
  }));
  return data;
}

export async function retrieveDataByID(collectionName: string, id: string) {
  const snapshot = await getDoc(doc(db, collectionName, id));
  const data = snapshot.data();
  return data;
}

export async function signUp(
  userData: {
    email: string;
    fullname: string;
    password: string;
  },
  callback: Function,
) {

  const q = query(
    collection(db, "users"),
    where("email", "==", userData.email),
  );
  const querySnapshot = await getDocs(q);
  const data = querySnapshot.docs.map((doc) => ({
    id: doc.id,
    ...doc.data(),
  }));

  if (data.length > 0) {
    // user sudah ada -> tidak boleh daftar
    callback({
      status: "error",
      message: "User already exists",
    });
  } else {
    // user belum ada -> boleh daftar
    await addDoc(collection(db, "users"), userData);
    callback({
      status: "success",
      message: "User registered successfully",
    });
  }
}
```

- Buat file register.ts pada folder api



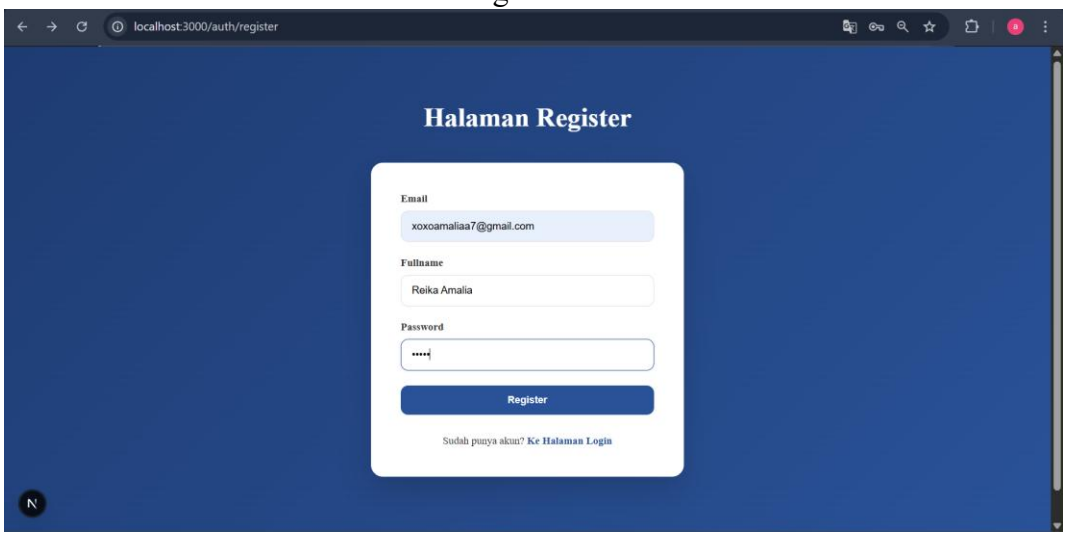
- Modifikasi file register.ts

```
// Next.js API route support: https://nextjs.org/docs/api-routes/introduction
import { signUp } from '@/utils/db/servicefirebase'
import type { NextApiRequest, NextApiResponse } from 'next'

type Data = {
  name: string
  alamat: string
}

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse<Data>
) {
  if(req.method === "POST"){
    await signUp(req.body, (result: {status: string, message: string}) => {
      if(result.status === "success"){
        res.status(200).json({ name: result.message, alamat: "" });
      } else {
        res.status(400).json({ name: result.message, alamat: "" });
      }
    });
  }
  else {
    res.status(405).json({ name: "Method not allowed", alamat: "" });
  }
}
```

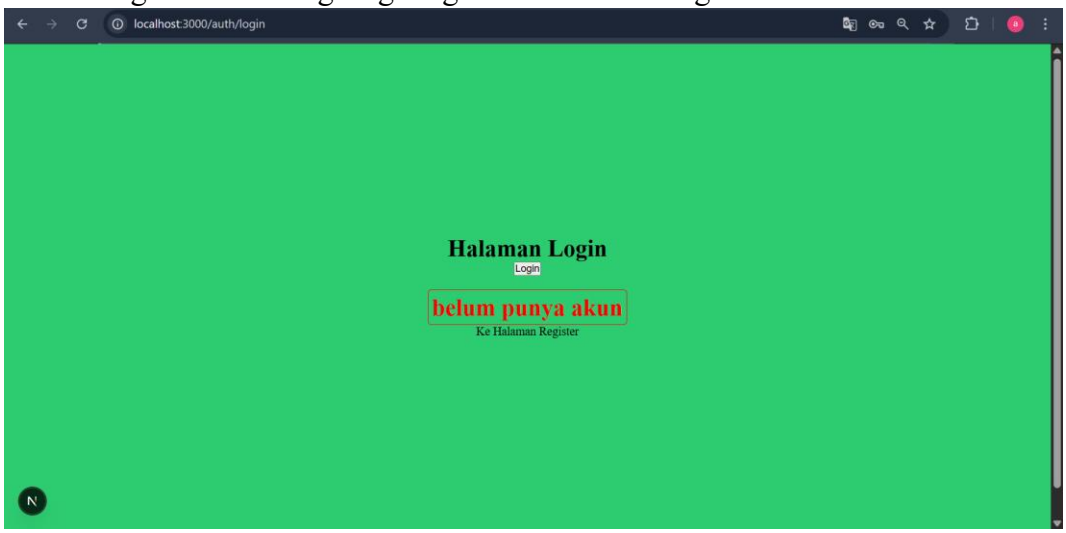
- Modifikasi index.tsx pada folder register (tambahkan beberapa code)

```
import { useState } from "react";
import { useRouter } from "next/router";

const TampilanRegister = () => {
  const [isLoading, setIsLoading] = useState(false);
  const { push } = useRouter();
  const [error, setError] = useState("");

  const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault();
    const form = event.currentTarget;
    const formData = new FormData(event.currentTarget);
    const email = formData.get("email") as string;
    const fullname = formData.get("Fullname") as string;
    const password = formData.get("Password") as string;
    const response = await fetch("/api/register", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ email, fullname, password }),
    });
    // const result = await response.json();
    // console.log(result);
    if (response.status === 200) {
      form.reset();
      // event.currentTarget.reset();
      setIsLoading(false);
      push("/auth/login");
    } else {
      setIsLoading(false);
      setError(
        response.status === 400 ? "User already exists" : "An error occurred",
      );
    }
  };

  return (
    <div className={style.register}>
      <h1 className={style.register__title}>Halaman Register</h1>

      <div className={style.register__form}>

        <form onSubmit={handleSubmit}>
```

- Buka browser http://localhost:3000/auth/register isikan data dan klik register. Jika berhasil maka akan masuk ke menu login



Setelah register akan langsung berganti ke halaman login secara otomatis.



Bagian 3 – Install bcrypt

- npm install bcrypt –force

```
PS D:\Pemograman_Framework\praktikum_15\my-app-15> npm install bcrypt --force
npm WARN using --force Recommended protections disabled.

added 3 packages, and audited 450 packages in 5s

148 packages are looking for funding
  run `npm fund` for details

1 moderate severity vulnerability

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
```

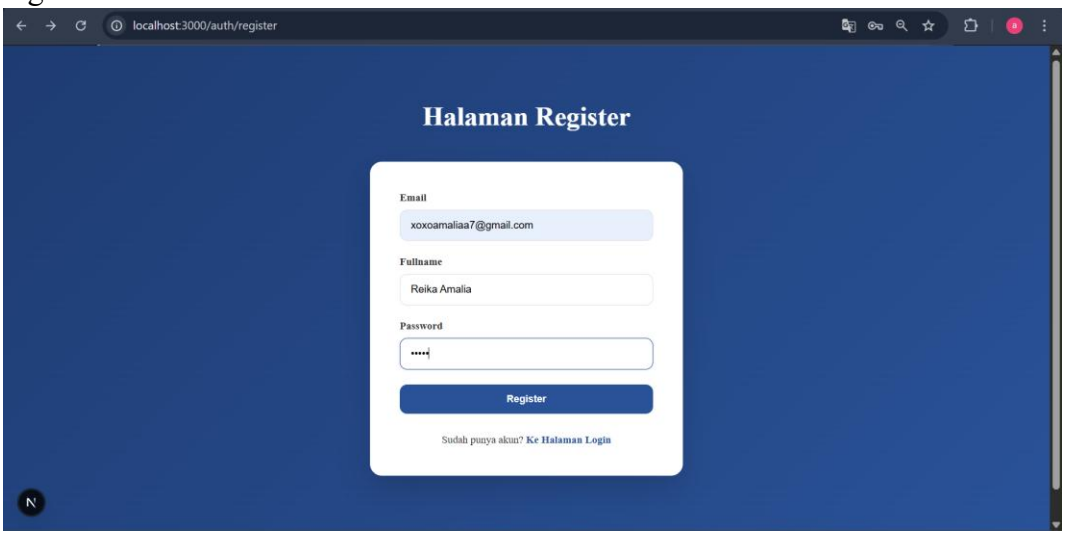
- npm install --save-dev @types/bcrypt –force

```
PS D:\Pemograman_Framework\praktikum_15\my-app-15> npm install --save-dev @t
ypes/bcrypt --force
npm WARN using --force Recommended protections disabled.

added 1 package, and audited 451 packages in 3s

148 packages are looking for funding
  run `npm fund` for details

1 moderate severity vulnerability

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
```

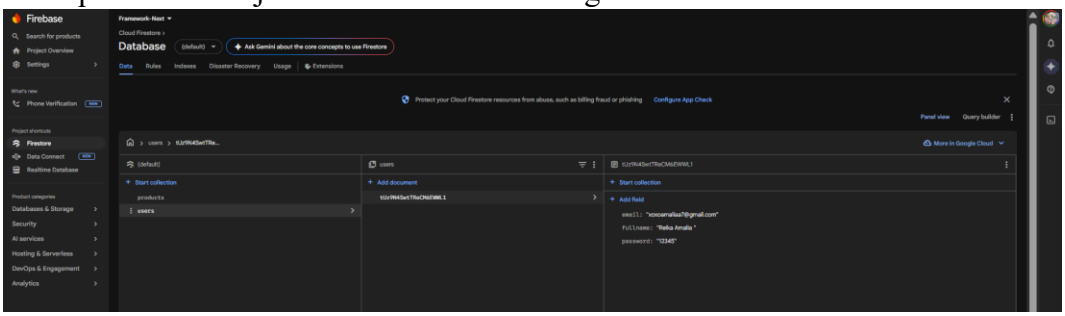
- Buka file servicefirebase.ts pada folder src/utils/db dan modifikasi

```python
import {
  getFirestore,
  collection,
  getDocs,
  getDoc,
  doc,
  query,
  addDoc,
  where,
} from "firebase/firestore";
import app from "./firebase";
import bcrypt from "bcrypt";

const db = getFirestore(app);

export async function retrieveProducts(collectionName: string) {

  // const snapshot = await getDocs(collection(db, collectionName));
  // const data = snapshot.docs.map((doc) => ({
  //   id: doc.id,
  //   ...doc.data(),
  // }));
  // return data;
}

export async function retrieveDataByID(collectionName: string, id: string) {
}

// export async function retrieveDataByID(collectionName: string, id: string) {
//   const snapshot = await getDoc(doc(db, collectionName, id));
//   const data = snapshot.data();
//   return data;
// }

export async function signUp(
  userData: {
    email: string;
    fullname: string;
    password: string;
    role?: string;
},
  callback: Function,
) {

  const q = query(
    collection(db, "users"),
    where("email", "==", userData.email),
  );
  const querySnapshot = await getDocs(q);
  const data = querySnapshot.docs.map((doc) => ({
    // id: doc.id,
    // ...doc.data(),
  }));

  if (data.length > 0)
    {
    // user sudah ada -> tidak boleh daftar
  //   callback({
  //     status: "error",
  //     message: "User already exists",
  //   });
  // } else {
    // user belum ada -> boleh daftar
    // await addDoc(collection(db, "users"), userData);

    callback({
      status: "success",
      message: "User registered successfully",
    });
    } else {
      userData.password = await bcrypt.hash(userData.password, 10);
      userData.role = "user";
      await addDoc(collection(db, "users"), userData)
        .then(() => {
          callback({
            status: "success",
            message: "User registered successfully",
          });
        })
        .catch((error) => {
          callback({
            status: "error",
            message: error.message,
          });
        });
          }
        }

```

- Jalankan browser http://localhost:3000/auth/register dan input data setelah itu klik register



- Buka pada firebase jika berhasil maka data register akan masuk



- Jika user memasukkan data yang sama sistem tidak akan memproses tetapi permasalahannya user memasukkan data yang sama tidak ada pemberitahuan pada layar maka dari itu perlu ada perubahan pada code index.tsx pada folder

  views/auth/register

  o Line 41

```
1  return (
2      <div className={style.register}>
3        {/* modifikasi line 41 */}
4        {error && <p className={style.register__error}>{error}</p>}
5        <h1 className={style.register__title}>Halaman Register</h1>
6
```

o Line 94 dan 96

```
1  {/* Button Register */}
2  <button
3    type="submit"
4    className={style.register__form__item__button}
5    disabled={isLoading}
6  >
7    {isLoading ? "Loading..." : "Register"}
8  </button>
```

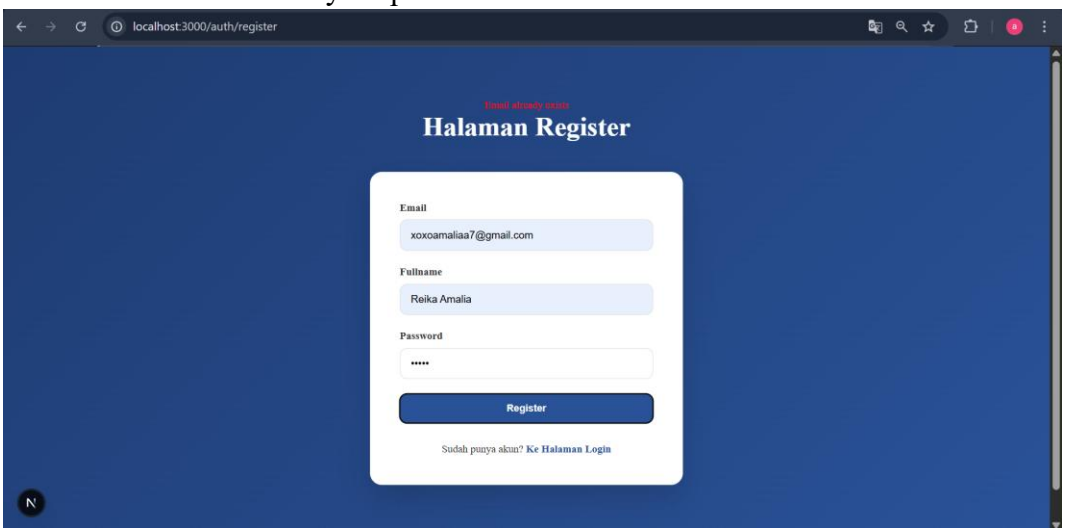
o Line 34 rubah menjadi email

```
1  if (response.status === 200) {
2    form.reset();
3    // event.currentTarget.reset();
4    setIsLoading(false);
5    push("/auth/login");
6  } else {
7    setIsLoading(false);
8    setError(
9      response.status === 400 ? "Email already exists" : "An error occurred",
10   );
```

- Modifikasi juga pada register.module.scss

```
my-app-15 > src > views > auth > register >
1    .register {
19     &__form {
27       &__item {
70         &__text {
85         }
86       }
87
88       &__error {
89         color: red;
90         font-size: 14px;
91         margin-top: 8px;
92       }
93     }
94   }
```

- Jika berhasil maka hasilnya seperti berikut



- Tambakan loading dengan menambahkan kode pada index.tsx

```
my-app-15 > src > views > auth > register > index.tsx > TampilanRegister > handl
1   import Link from "next/link";
2   import style from "../../auth/register/register.module.scss";
3   import { useState } from "react";
4   import { useRouter } from "next/router";
5
    Windsurf: Refactor | Explain | Generate JSDoc | X
6   const TampilanRegister = () => {
7     const [isLoading, setIsLoading] = useState(false);
8     const { push } = useRouter();
9     const [error, setError] = useState("");
10
      Windsurf: Refactor | Explain | Generate JSDoc | X
11    const handleSubmit = async (event: React.FormEvent<HTMLFor
12      setError("");
13      setIsLoading(true);
```

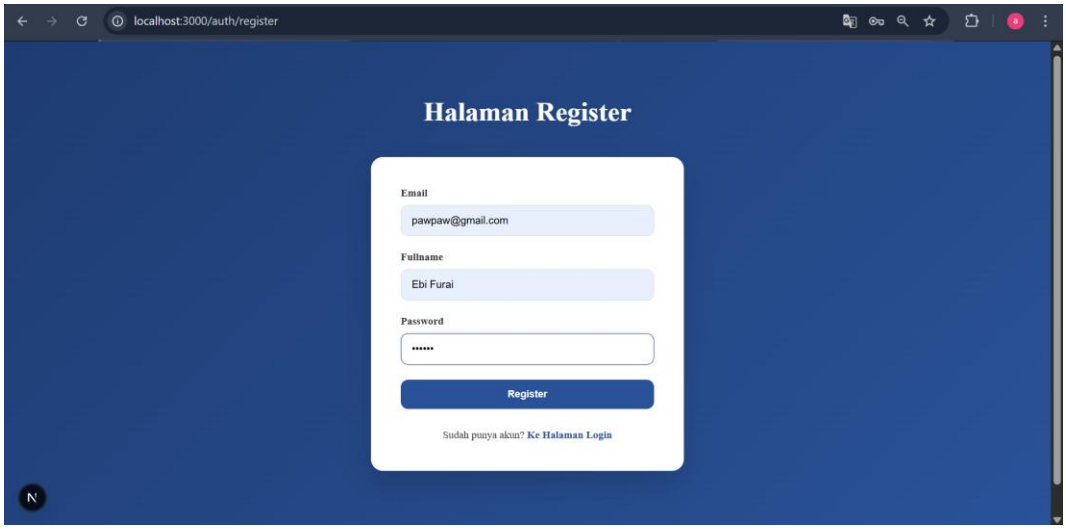
- Jika berhasil maka hasilnya akan muncul loading saat klik register

D. Pengujian

Uji 1 – Register Baru

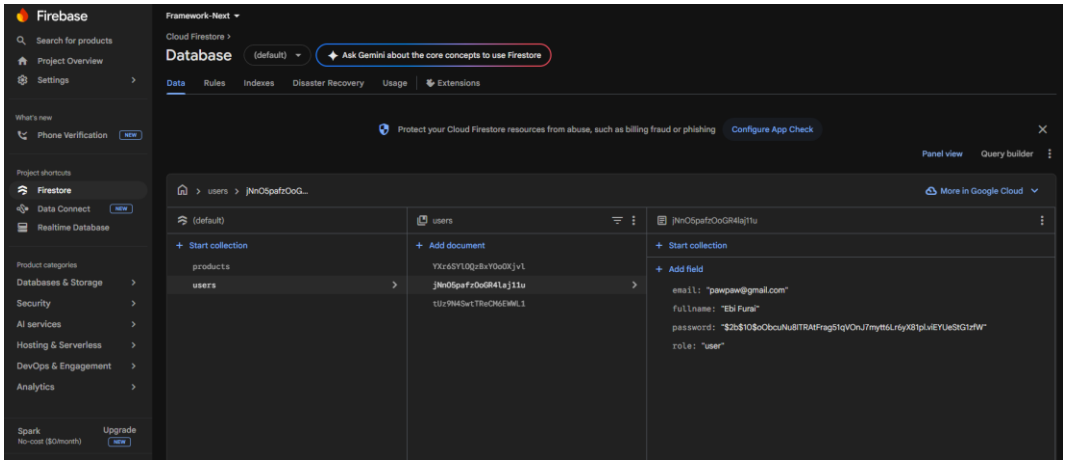
Input:

- Email baru

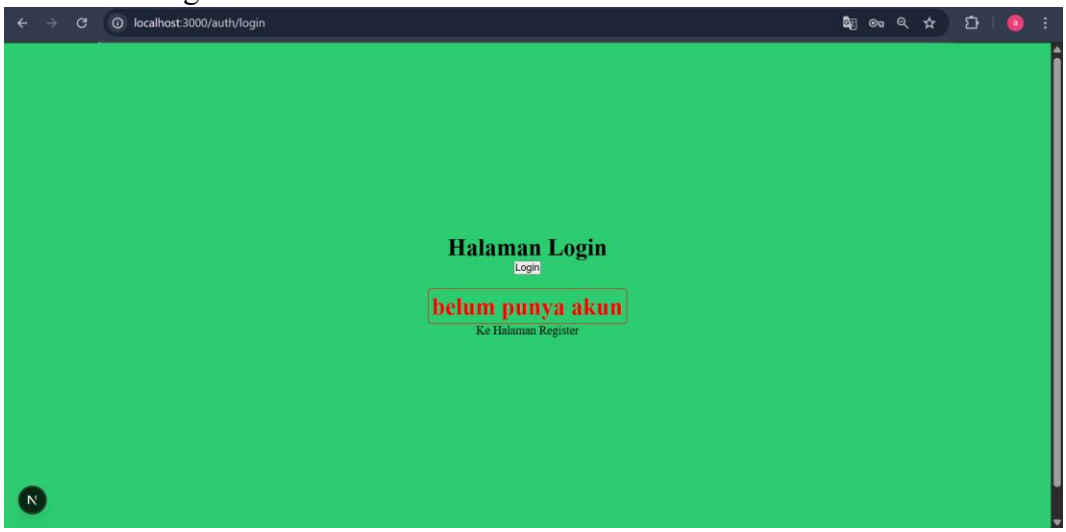


Hasil:

- Data tersimpan di Firestore
- Password ter-hash



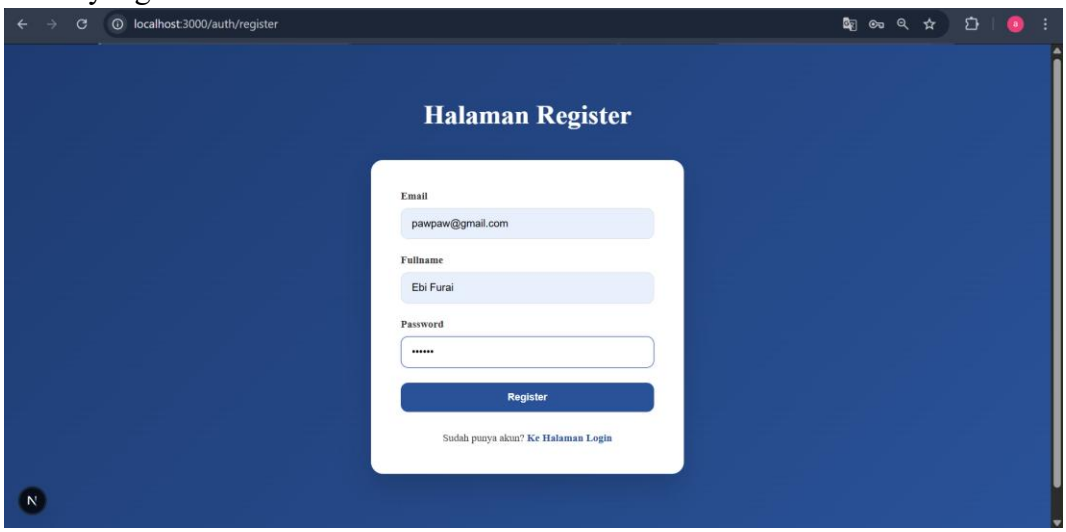
- Redirect ke login



Uji 2 – Email Sudah Ada

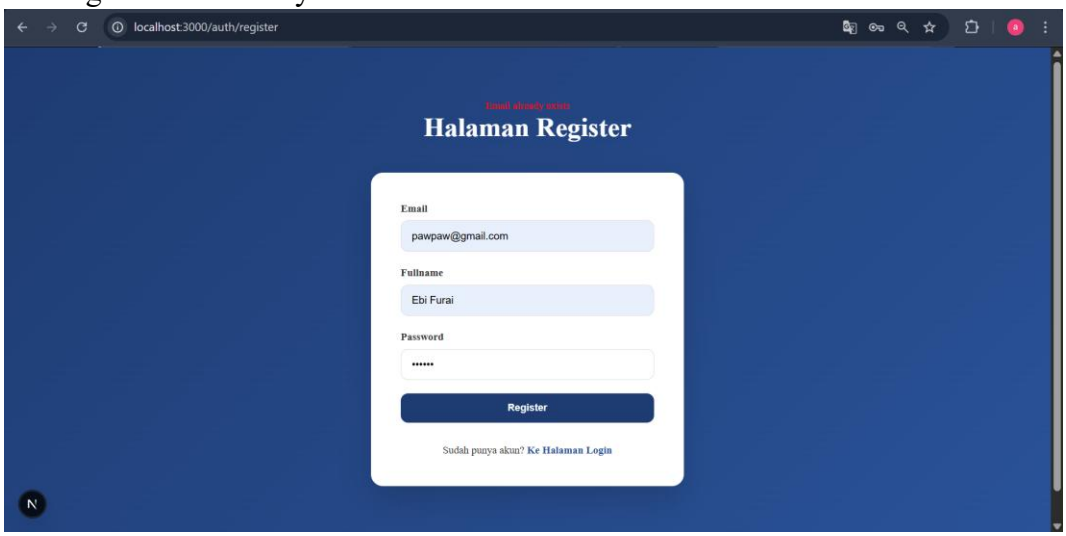
Input:

- Email yang sama



Hasil:

- Error 400
- Message: Email already exists



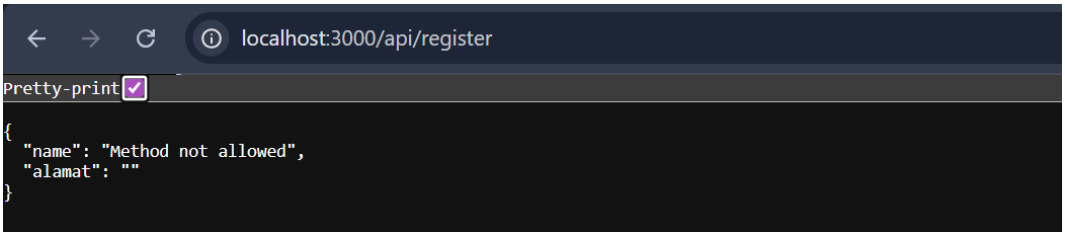
Uji 3 – Method GET

Akses:

/api/register

Hasil:

- 405 Method Not Allowed



E. Struktur Database (Firestore)

Collection: users

Field:

Field	Tipe
fullName	string
email	string
password	string (hashed)
role	string
createdAt	timestamp

G. Tugas Praktikum

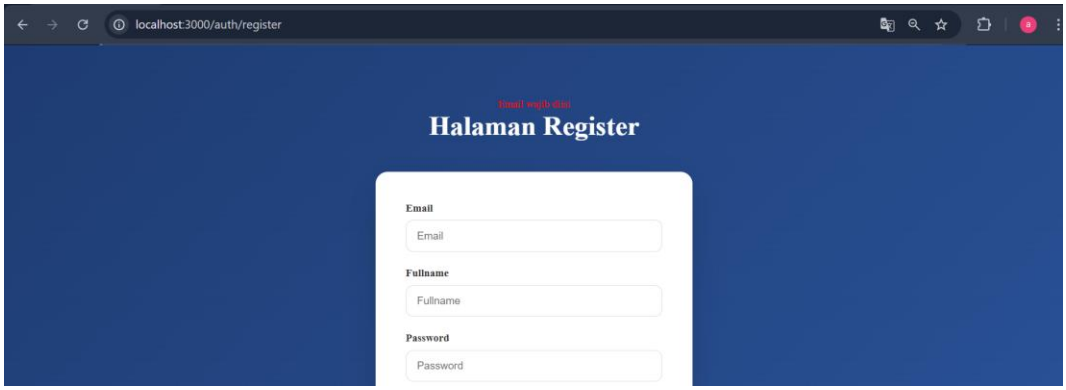
1. Implementasikan register terhubung database.

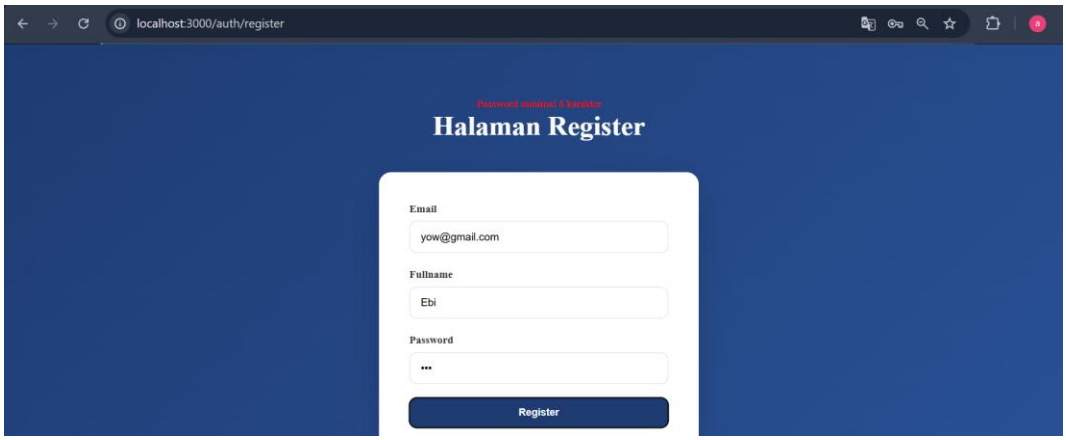
```
1  await addDoc(collection(db, "users"), userData)
```

2. Tambahkan validasi:

- Email wajib
- Password minimal 6 karakter

```
const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
  setError("");
  setIsLoading(true);
  event.preventDefault();
  const form = event.currentTarget;
  const formData = new FormData(event.currentTarget);
  const email = formData.get("email") as string;
  const fullname = formData.get("Fullname") as string;
  const password = formData.get("Password") as string;

  // validasi email dan password
  if (!email) {
    setError("Email wajib diisi");
    setIsLoading(false);
    return; // ← stop, tidak lanjut ke fetch
  }
  if (password.length < 6) {
    setError("Password minimal 6 karakter");
    setIsLoading(false);
    return; // ← stop, tidak lanjut ke fetch
  }
```

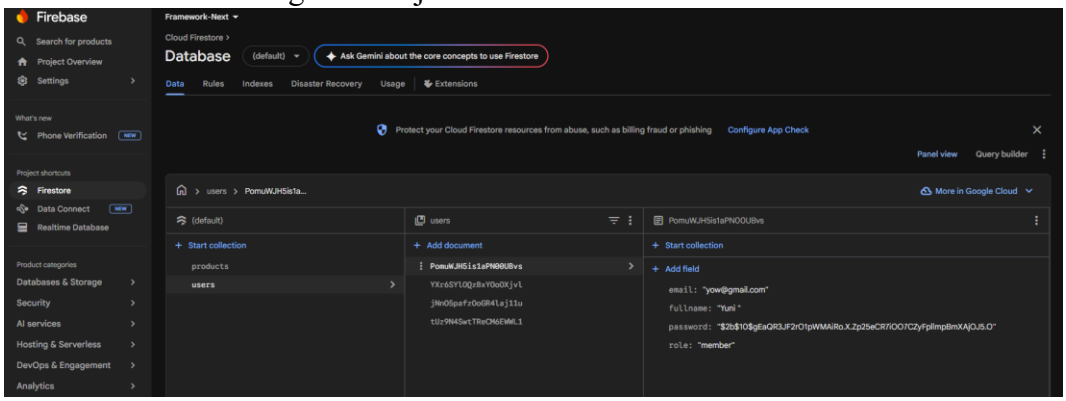




3. Tambahkan role default "member".

```
1  } else {
2        userData.password = await bcrypt.hash(userData.password, 10);
3        userData.role = "member";
4        await addDoc(collection(db, "users"), userData)
```

Default role sudah berganti menjadi member

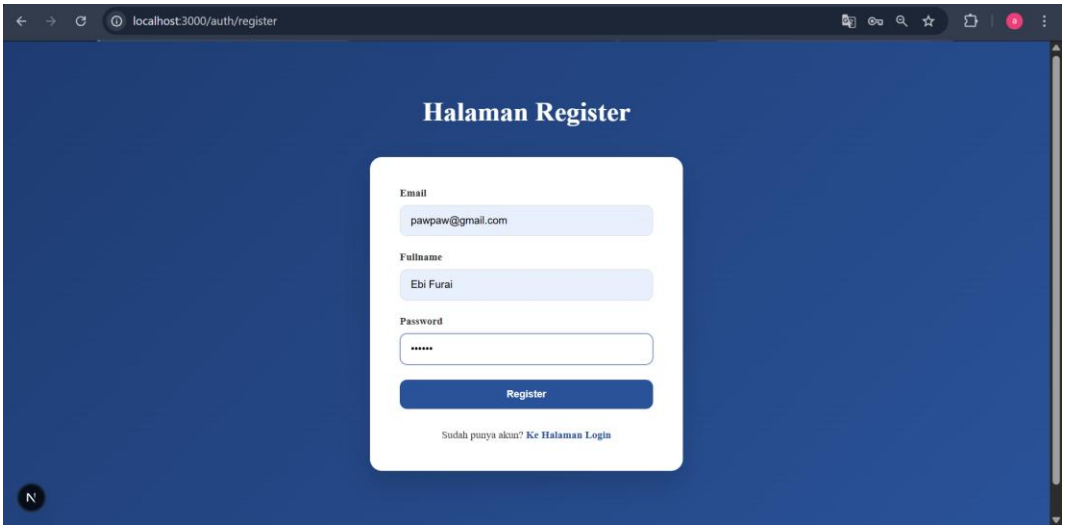


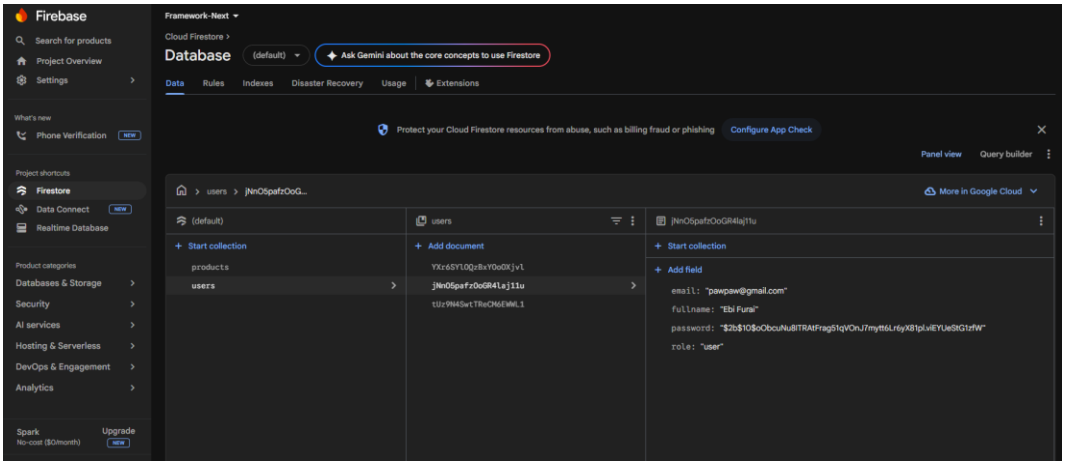
4. Tampilkan pesan error di UI.

```
1  return (
2      <div className={style.register}>
3        {/* modifikasi line 41 */}
4        {error && <p className={style.register__form__error}>{error}</p>}
5        <h1 className={style.register__title}>Halaman Register</h1>
6
```

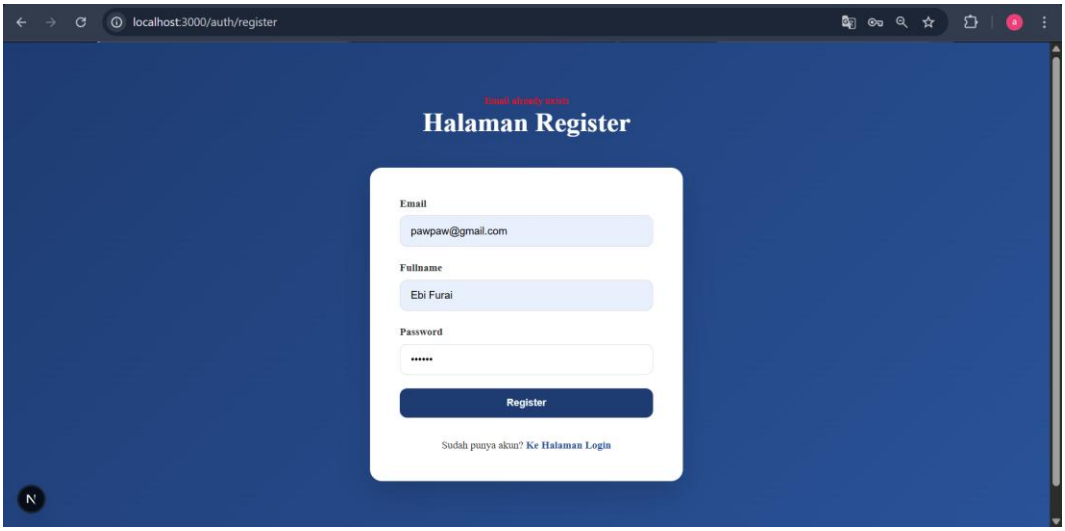
5. Screenshot hasil:

- Register sukses

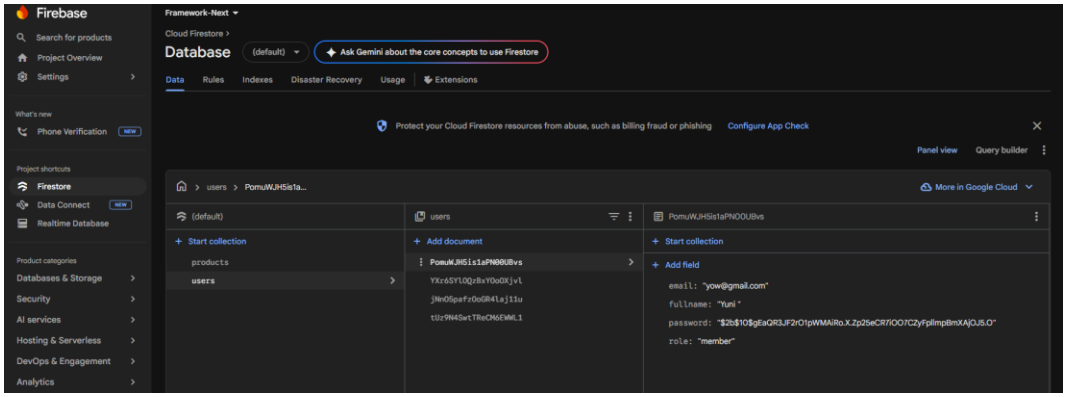




- Email sudah ada



- Database Firestore



H. Pertanyaan Analisis

1. Mengapa password harus di-hash?
   - Password harus di-hash untuk menjaga keamanan data pengguna. Dengan hashing, password tidak disimpan dalam bentuk asli, sehingga jika database bocor, password tidak bisa langsung diketahui. Hal ini mencegah penyalahgunaan akun oleh pihak yang tidak bertanggung jawab.

2. Apa perbedaan addDoc dan setDoc?
   - addDoc digunakan untuk menambahkan data baru dengan ID yang dibuat secara otomatis oleh Firebase. Sedangkan setDoc digunakan untuk menyimpan data dengan ID yang sudah ditentukan sebelumnya. Jadi, setDoc lebih fleksibel jika ingin mengatur ID sendiri.

3. Mengapa perlu validasi method POST?
   - Validasi method POST diperlukan agar endpoint hanya menerima request sesuai tujuan, yaitu untuk menambahkan data. Hal ini mencegah kesalahan penggunaan API, misalnya penggunaan GET untuk mengirim data, serta meningkatkan keamanan sistem.

4. Apa risiko jika email tidak dicek unik?
   - Jika email tidak dicek unik, maka bisa terjadi satu email digunakan oleh beberapa akun. Hal ini dapat menyebabkan kebingungan dalam proses login dan berpotensi menimbulkan masalah keamanan, seperti akses akun yang tidak jelas kepemilikannya.

5. Apa fungsi role pada user?
   - Role digunakan untuk membedakan hak akses setiap user dalam sistem. Misalnya admin memiliki akses penuh, sedangkan user biasa hanya memiliki akses terbatas. Dengan adanya role, sistem dapat mengatur otorisasi secara lebih terstruktur dan aman.